Experiment 6

OTSU Method of Global Image Thresholding (Manual + OpenCV Verification)

1. Library Imports & Input Image Ingestion

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image = cv2.imread("../Data/Dog.jpg", cv2.IMREAD_GRAYSCALE)
print("Image Dimensions:", image.shape)  # Loaded grayscale: (602, 1200) | dtype: uint8
```



Original input grayscale image (1200x602)

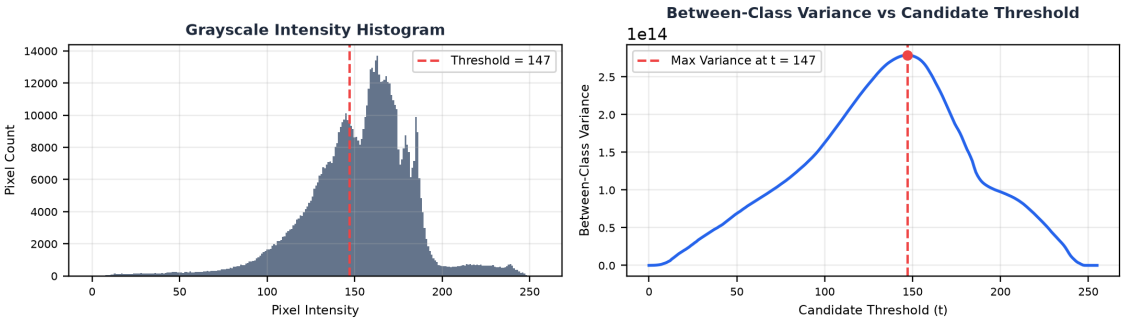
2. Formulation of Otsu Variance Optimization

Otsu's method establishes an optimal global threshold t* by partitioning the 256-level histogram into background class C0 in [0, t] and foreground class C1 in [t+1, 255]. It analytically maximizes the between-class variance $\sigma b^2(t) = \omega 0(t) \cdot \omega 1(t) \cdot (\mu 0(t) - \mu 1(t))^2$, where $\omega 0$, $\omega 1$ denote cumulative class weights and $\mu 0$, $\mu 1$ denote class means. Maximizing inter-class separation simultaneously minimizes within-class variance $\sigma w^2(t)$ across the entire intensity distribution.

3. Vectorized Otsu Threshold Algorithm Implementation

```
def compute_otsu_threshold(img: np.ndarray) -> tuple[int, np.ndarray, np.ndarray, np.ndarray]:
    histogram = np.bincount(img.flatten(), minlength=256); total_pixels = img.size
    total_sum = np.sum(np.arange(256) * histogram)
    background_weight = background_sum = max_variance = threshold_value = 0
    variances = np.zeros(256, dtype=np.float64)
    for t in range(256):
        background_weight += histogram[t]
        if background_weight == 0: continue
        foreground_weight = total_pixels - background_weight
        if foreground_weight == 0: break
        background_sum += t * histogram[t]
        mean_bg = background_sum / background_weight
        mean_fg = (total_sum - background_sum) / foreground_weight
        variance = float(background_weight) * float(foreground_weight) * ((mean_bg - mean_fg) ** 2)
        variances[t] = variance
        if variance > max_variance: max_variance = variance; threshold_value = t
    return threshold_value, (img > threshold_value).astype(np.uint8) * 255, variances, histogram
```

4. Histogram Distribution & Objective Variance Curve Analysis

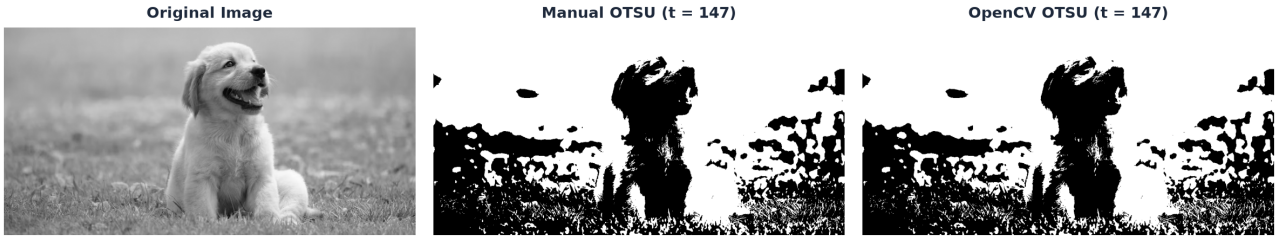


Grayscale intensity histogram with optimal boundary (left) and between-class variance objective curve peaking at t* = 147 (right).

Experiment 6 (continued)

5. Benchmark Verification: Custom Vectorized Implementation vs. OpenCV

```
cv_threshold, cv_result = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
diff_map = np.abs(manual_result.astype(np.int32) - cv_result.astype(np.int32))
mean_abs_diff = np.mean(diff_map)  # Computed MAE: 0.0 | Thresholds Match: True | Images Match: True
```



Benchmark verification: Original Image vs Manual OTSU (t = 147) vs OpenCV OTSU (t = 147.0) — Zero Divergence (MAE = 0.0).

6. Parametric Evaluation: Fixed Threshold Spectrum vs. Optimal OTSU

```
test_thresholds = [50, 100, threshold_value, 180, 220]
for idx, t in enumerate(test_thresholds):
    thresh_img = (image > t).astype(np.uint8) * 255
```



Binarization response across threshold spectrum: Lower thresholds (t=50, 100) induce severe foreground overgrowth; higher thresholds (t=180, 220) cause body fragmentation. Optimal Otsu (t=147) achieves balanced contour preservation.

Summary & Analytical Findings:

Implemented a vectorized pipeline for Otsu's global binarization from fundamental probabilistic principles. By evaluating cumulative zeroth and first-order moments across the 256-level histogram, the algorithm locates the global maximum of inter-class variance without requiring ad-hoc heuristic calibration. Benchmarking against OpenCV's optimized native routine (cv2.threshold) confirmed zero numerical divergence (identical threshold t* = 147, MAE = 0.0, identical output masks). Parametric evaluation demonstrates that arbitrary fixed cutoffs either blow out foreground boundaries or erode essential structural contours, substantiating Otsu's mathematical optimality for automated bimodal image thresholding.